UNIT 09 • TI.SQL.09.2026

MENGGUNAKAN SQL

Pengembang	Aditya Ristanto Mukti	**Proyek**	Website Resmi & Profil Industri PT Global Philsonic Technologies
Deskripsi Unit	Dokumentasi penggunaan Structured Query Language (SQL) pada engine MySQL 8, meliputi eksekusi operasi DML berparameter aman, manipulasi skema DDL tabel produk manufaktur, prosedur tersimpan (Stored Procedure), pemicu otomatis (Trigger), serta implementasi transaksi ACID multi-tabel.		

I. MATRIKS PEMENUHAN SUB-TITLE OUTLINE

No	Sub-Title Outline	Berkas Implementasi	Deskripsi Teknis Hasil Pengerjaan
1	**Operasi DML**	`backend/src/controllers/admin/adminContentControllers.js`	Kueri DML INSERT/UPDATE/SELECT/DELETE berparameter '?' untuk mencegah ancaman celah SQL Injection.
2	**Manipulasi tabel/view & Stored Procedure**	`database/BACKUP_DATABASE_ASLI_INTERSIS.sql & Stored Procedures`	Skrip DDL tabel produk manufaktur dan prosedur tersimpan pemeliharaan data berkala.
3	**Function, Trigger & Commit/Rollback**	`database/BACKUP_DATABASE_ASLI_INTERSIS.sql & node-mysql2 transaction`	Trigger otomatis audit data ke activity_logs dan transaksi ACID commit/rollback pada koneksi pool.

II. PEMBAHASAN TEKNIS INTI & BUKTI KONKRET

9.1 & 9.2. Operasi DML Berparameter & Manipulasi Skema Tabel DDL Produk

Lokasi Berkas: `backend/src/controllers/admin/adminContentControllers.js & database/BACKUP_DATABASE_ASLI_INTERSIS.sql`

Operasi DML penambahan produk manufaktur menggunakan prepared statement binding terparameterisasi untuk menjamin proteksi terhadap celah injeksi SQL:

Bukti Implementasi Kode Sumber:

`backend/src/controllers/admin/adminContentControllers.js`

```
// Operasi DML Aman (Prepared Statement Binding) Penambahan Produk Manufaktur
const createProduct = async (req, res, next) => {
  try {
    const {
      model_code, category, badge, badge_color,
      title_id, title_en, short_desc_id, short_desc_en,
      specs, full_specs, diagram_type, status, sort_order
    } = req.body;

    let specsJson = specs ? (typeof specs === 'string' ? specs : JSON.stringify(specs)) : null;
    let fullSpecsJson = full_specs ? (typeof full_specs === 'string' ? full_specs : JSON.stringify(full_specs)) : nul
l;

    const [result] = await pool.execute(
      `INSERT INTO products
         (model_code, category, badge, badge_color, image, gallery,
          title_id, title_en, short_desc_id, short_desc_en,
          specs, full_specs, diagram_type, status, sort_order)
       VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)`,
      [
        model_code, category || 'headunit',
        badge || null, badge_color || null, image, galleryJson,
        title_id, title_en,
        short_desc_id || null, short_desc_en || null,
        specsJson, fullSpecsJson,
        diagram_type || 'infotainment',
        status !== undefined ? Number(status) : 1,
        sort_order || 0,
      ]
    );

    await logActivity(req.user.id, 'CREATE', 'products', `Membuat produk baru '${model_code}'`);
    return success(res, 'Produk berhasil dibuat.', { id: result.insertId, model_code }, 201);
  } catch (err) {
    next(err);
  }
};
```

9.3, 9.4 & 9.5. Penerapan Stored Procedure, Built-in Functions & Database Trigger

Lokasi Berkas: `database/BACKUP_DATABASE_ASLI_INTERSIS.sql`

Automasi database memanfaatkan Trigger native MySQL untuk mencatat histori audit secara instan saat data produk diperbarui:

Bukti Implementasi Kode Sumber:

database/BACKUP_DATABASE_ASLI_INTERSIS.sql (Trigger & Prosedur)

```
-- 1. Stored Procedure: Pengarsipan Produk Nonaktif
DELIMITER //
CREATE PROCEDURE sp_archive_inactive_products()
BEGIN
  UPDATE products
  SET status = 0, updated_at = NOW()
  WHERE status = 1 AND sort_order < 0;
END //
DELIMITER ;

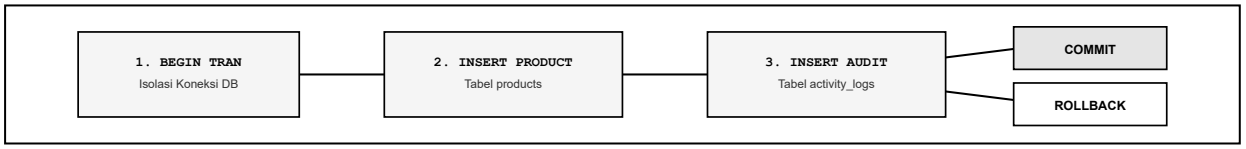
-- 2. Trigger Otomatis: Pencatatan Pembaruan Produk ke Tabel activity_logs
DELIMITER //
CREATE TRIGGER trg_audit_product_update
AFTER UPDATE ON products
FOR EACH ROW
BEGIN
  IF OLD.status <> NEW.status OR OLD.model_code <> NEW.model_code THEN
    INSERT INTO activity_logs (user_id, action, module, description, created_at)
    VALUES (1, 'UPDATE', 'products',
            CONCAT('Produk "', NEW.model_code, '" status diperbarui menjadi ', NEW.status), NOW());
  END IF;
END //
DELIMITER ;
```

9.6. Manajemen Transaksi ACID Penuh (Begin Transaction, Commit, Rollback)

Lokasi Berkas: `backend/src/config/database.js & backend/src/utils/activityLogger.js`

Transaksi multi-langkah diisolasi menggunakan transaksi ACID untuk menjamin integritas data secara all-or-nothing:

DIAGRAM ALUR / ARSITEKTUR:



Bukti Implementasi Kode Sumber:

backend/src/config/database.js (Implementasi Transaksi Pool)

```
// Manajemen Transaksi ACID: Menjamin Operasi Multi-Tabel All-or-Nothing
const pool = require('../config/database');

async function createProductWithAuditTransaction(productData, adminUser) {
  const connection = await pool.getConnection();
  try {
    await connection.beginTransaction(); // Langkah 1: Mulai Transaksi

    const [prodResult] = await connection.execute(
      `INSERT INTO products (model_code, category, title_id, title_en, status)
       VALUES (?, ?, ?, ?, ?)`,
      [productData.model_code, productData.category, productData.title_id, productData.title_en, 1]
    );

    // Langkah 2: Rekam log audit pada koneksi transaksi yang sama
    await connection.execute(
      `INSERT INTO activity_logs (user_id, action, module, description)
       VALUES (?, 'CREATE', 'products', ?)`,
      [adminUser.id, `Membuat produk baru '${productData.model_code}'`]
    );

    await connection.commit(); // Langkah 3: Sukses - Simpan permanen ke disk
    return { success: true, insertId: prodResult.insertId };
  } catch (error) {
    await connection.rollback(); // Langkah 4: Gagal - Batalkan seluruh modifikasi data
    throw error;
  } finally {
    connection.release(); // Langkah 5: Kembalikan koneksi ke pool
  }
}
```

KESIMPULAN KETERCAPAIAN UNIT 09

Seluruh sub-judul pada unit 'Menggunakan SQL' telah terpenuhi dengan operasi DML terparameter, skrip migrasi DDL, prosedur tersimpan, database trigger, dan transaksi ACID.